色块识别-彩色图

色块识别-彩色图

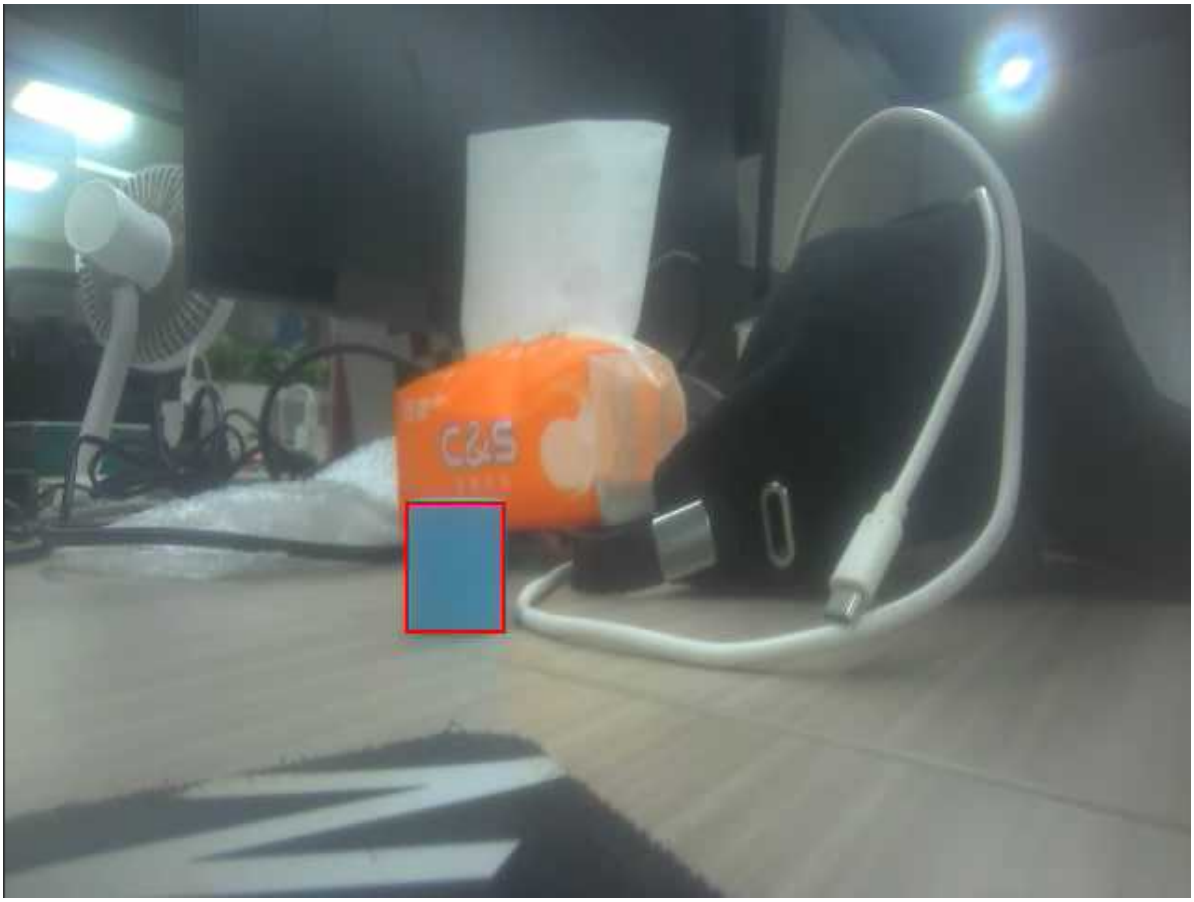
- 例程效果
- 代码讲解
  - 导入模块
    - 设置图像尺寸
    - 初始化摄像头（RGB888 模式）
    - 初始化显示模块
    - 初始化媒体管理器并启动摄像头
    - 设置彩色块检测参数
    - 图像处理与彩色块检测
    - 资源释放
  - 参数调整说明

例程效果

运行本节例程代码 [源码汇总/17.cv_lite/2.rgb888_find_blobs.py]

本节是基于RGB888（即彩色）图像的色块识别，K230会检测画面中阈值符合设定的色块，并用矩形框标记出来

> 观察串行终端输出的帧率信息，在供电稳定的情况下帧率可以稳定在55帧附近



代码讲解

导入模块

```
import time, os, sys, gc
from machine import Pin
from media.sensor import *  # 导入摄像头接口 / Camera interface
from media.display import * # 导入显示接口 / Display interface
from media.media import *   # 导入媒体资源管理器 / Media manager
import _thread
import cv_lite              # cv_lite 扩展模块
import ulab.numpy as np     # MicroPython NumPy 类库
```

这些模块提供了摄像头控制、图像显示、内存管理和彩色块检测所需的功能。

设置图像尺寸

```
image_shape = [480, 640]  # 高 x 宽 / Height x Width
```

定义图像的分辨率为 480x640（高度 x 宽度），后续会根据此尺寸初始化摄像头和显示模块。

初始化摄像头（RGB888 模式）

```
sensor = Sensor(id=2, fps = 90)
sensor.reset()
sensor.set_framesize(width=image_shape[1], height=image_shape[0])
sensor.set_pixformat(Sensor.RGB888)  # RGB888格式 / rgb888 format
```

- 初始化摄像头，设置帧率为 90 FPS。
- 摄像头使用默认分辨率
- 将输出帧大小调整为 640x480，并设置为 RGB888 格式（三通道彩色图像，适合颜色检测）。

初始化显示模块

```
Display.init(Display.ST7701, width=image_shape[1], height=image_shape[0],
to_ide=True, quality=50)
```

初始化显示模块，使用 LCD 驱动芯片，分辨率与图像尺寸一致。`to_ide=True` 表示图像会同时传输到 IDE 进行虚拟显示，`quality=50` 设置图像传输质量。

初始化媒体管理器并启动摄像头

```
MediaManager.init()
sensor.run()
```

初始化媒体资源管理器，并启动摄像头开始捕获图像流。

设置彩色块检测参数

```
threshold = [120, 255, 0, 50, 0, 50]  # 格式: [Rmin, Rmax, Gmin, Gmax, Bmin, Bmax]
min_area = 100    # 最小色块面积 / Minimum blob area
kernel_size = 1   # 腐蚀膨胀核大小（用于预处理）/ Kernel size for morphological ops
clock = time.clock()  # 启动帧率计时器 / Start FPS timer
```

- `threshold`：颜色检测阈值范围，格式为 `[Rmin, Rmax, Gmin, Gmax, Bmin, Bmax]`，表示 RGB 三个通道的取值范围。此处设置为检测红色区域（R 通道范围为 120-255，G 和 B 通道范围为 0-50）。
- `min_area`：连通域的最小面积，小于此值的区域会被忽略。
- `kernel_size`：图像腐蚀和膨胀操作的核大小，用于预处理图像以去除噪声。
- `clock`：用于计算帧率。

图像处理与彩色块检测

```
while True:
    clock.tick()

    # 拍摄一帧图像 / Capture a frame
    img = sensor.snapshot()
    img_np = img.to_numpy_ref()  # 获取 RGB888 ndarray 引用

    # 调用 cv_lite 扩展进行色块检测，返回 [x, y, w, h, ...] 列表
    blobs = cv_lite.rgb888_find_blobs(image_shape, img_np, threshold, min_area,
kernel_size)

    # 遍历检测到的色块并绘制矩形框
    for i in range(len(blobs) // 4):   # 修正为整数除法
        x = blobs[4*i]
        y = blobs[4*i + 1]
        w = blobs[4*i + 2]
        h = blobs[4*i + 3]
        img.draw_rectangle(x, y, w, h, color=(255, 0, 0), thickness=2)  # 红色框

    # 显示结果图像 / Show image with blobs
    Display.show_image(img)

    # 打印帧率 / Print FPS
    print("findblobs:", clock.fps())

    # 垃圾回收 / Garbage collect
    gc.collect()
```

- **图像捕获**：通过 `sensor.snapshot()` 获取一帧图像，并通过 `to_numpy_ref()` 将其转换为 NumPy 数组引用。
- **彩色块检测**：调用 `cv_lite.rgb888_find_blobs()` 函数进行彩色块检测，返回检测到的区域信息列表（每个 Blob 占用 4 个元素：`[x, y, w, h]`，即左上角坐标和宽度、高度）。
- **绘制矩形**：遍历检测到的每个 Blob，在图像上绘制一个红色矩形框（`color=(255, 0, 0)`）标注目标区域。
- **显示图像**：将处理后的图像显示到屏幕或 IDE 虚拟窗口。
- **内存管理和帧率输出**：调用 `gc.collect()` 清理内存，并通过 `clock.fps()` 打印当前帧率。

资源释放

```
sensor.stop()
Display.deinit()
os.exitpoint(os.EXITPOINT_ENABLE_SLEEP)
time.sleep_ms(100)
MediaManager.deinit()
```

程序退出时，停止摄像头、释放显示模块资源，并清理媒体管理器。

参数调整说明

- `threshold`：根据目标颜色调整 RGB 通道的阈值范围。例如：
  - 检测红色：`[120, 255, 0, 50, 0, 50]`（R 高，G 和 B 低）。
  - 检测绿色：`[0, 50, 120, 255, 0, 50]`（G 高，R 和 B 低）。
  - 检测蓝色：`[0, 50, 0, 50, 120, 255]`（B 高，R 和 G 低）。
- `min_area`：根据目标大小调整此值，较小的值会检测到更多小区域，较大的值会过滤掉噪声。
- `kernel_size`：用于图像腐蚀和膨胀，值越大，降噪效果越强，但可能丢失小目标的细节。